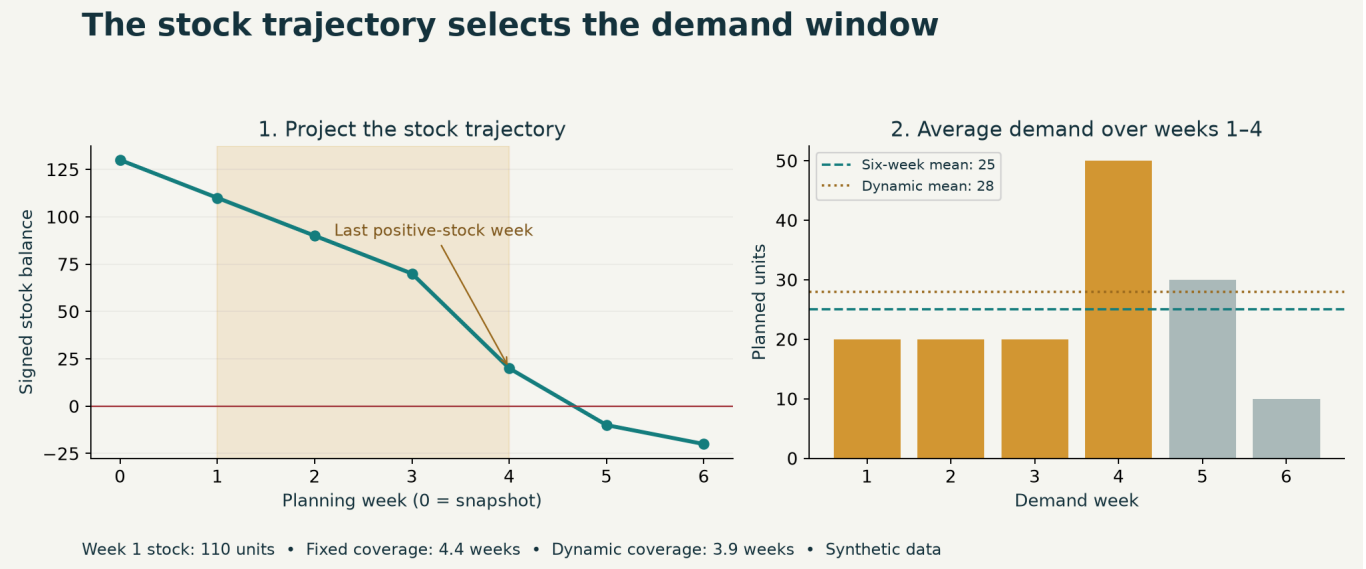
Weeks of Stock: turning a demand plan into a useful KPI

How much inventory coverage do we have when demand changes from week to week?

I built this dbt pipeline to connect stock snapshots, planned demand and incoming purchases, project the stock balance recursively, and calculate coverage using a demand window that follows the projected depletion horizon.

The important modelling decision is the order of those calculations: **demand determines the stock trajectory; the trajectory determines the averaging window.** The dynamic average expresses coverage. It never drives the forecast it was derived from.



This portfolio version starts from my existing SQL and YAML. Reusable models have moved upstream into an intermediate layer; the recursive calculation and KPI definitions are retained. All input records and the example results are synthetic. The local dbt project runs with DuckDB so a reviewer can inspect it without a warehouse account.

Explaining the KPI to a stakeholder

> We start with the stock we have and work forward week by week, adding deliveries and subtracting planned demand. If stock has a short projected horizon, we calculate average demand over that shorter period. Otherwise we use six weeks. We then express stock in weeks using that average. This makes the denominator relevant to the period the stock is expected to cover.

This is useful when demand is uneven: the average over six weeks can describe a different pattern from the weeks relevant to a short-stock item. A new delivery can also change the horizon, so it can change which demand weeks contribute to the average.

The output is a planning measure with explicit rules. It is not a recommendation to order a particular quantity, and it does not turn inventory availability into a new demand forecast.

A worked example

Take `SHORT_A` across all warehouses. Opening stock is **130 units**, with no incoming purchases. The first six demand weeks are **20, 20, 20, 50, 30, 10**.

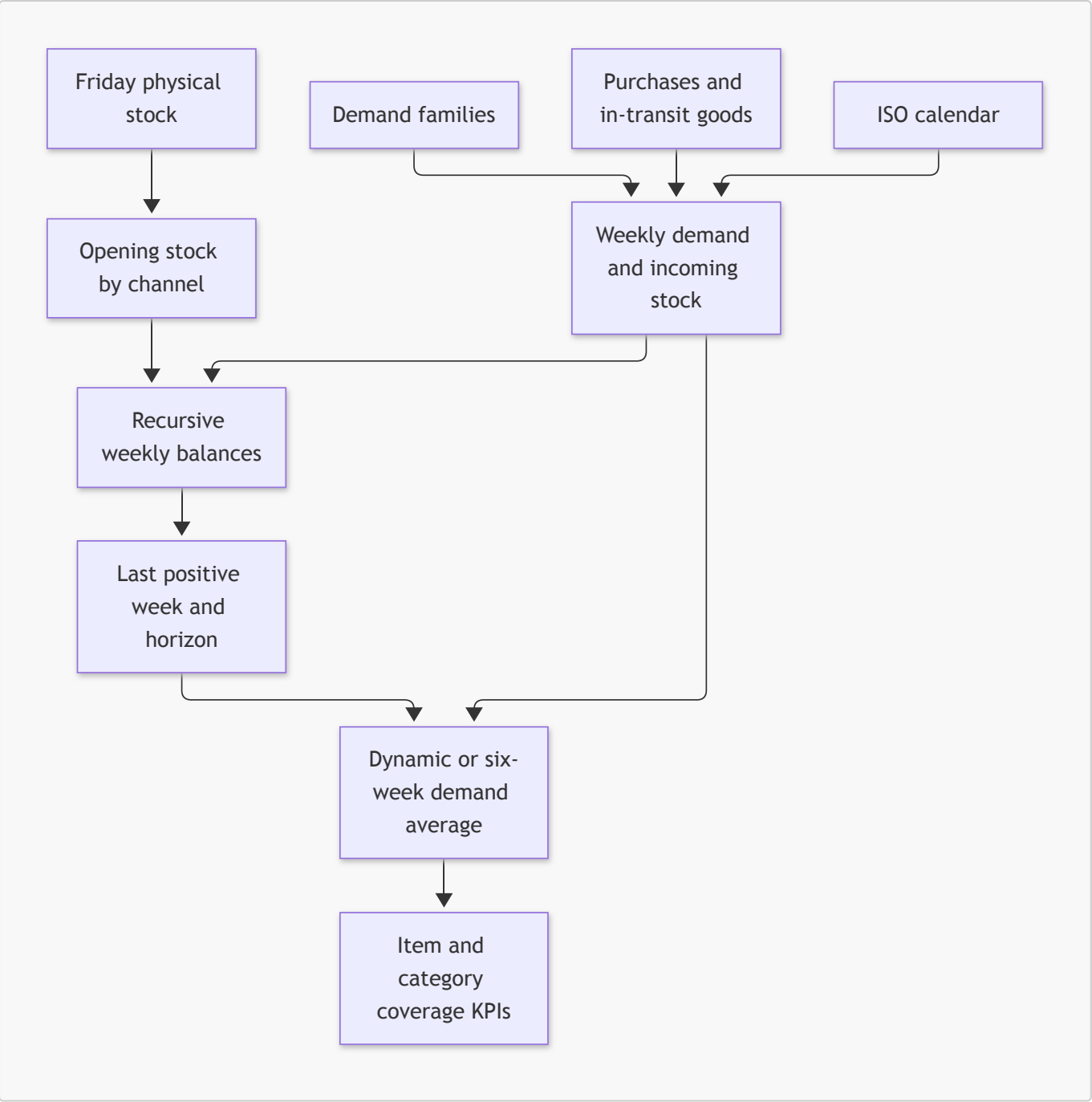
Planning position	Snapshot	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Planned demand	—	20	20	20	50	30	10
Signed closing stock	130	110	90	70	20	-10	-20

The last week with positive stock is **week 4**. At the week 1 reporting row:

Calculation	Result
Six-week average: `(20 + 20 + 20 + 50 + 30 + 10) / 6`	25 units/week
Distance from week 1 to the last positive week	3 weeks
Inclusive dynamic window	Weeks 1–4: 4 observations
Dynamic average: `ROUND((20 + 20 + 20 + 50) / 4, 0)`	28 units/week
Six-week coverage: `ROUND(110 / 25, 1)`	4.4 weeks
Dynamic coverage: `ROUND(110 / 28, 1)`	3.9 weeks

Those are two definitions applied to the same stock, not two different demand forecasts. The dynamic measure is selected because the projected horizon is below six weeks. The metric contract spells out the baseline convention, inclusivity and rounding.

The modelling behind it



The category branch aggregates stock, demand and purchases **before running its own recursion**. It does not average item-level weeks-of-stock ratios. Category coverage is a pooled planning view; it cannot guarantee availability of every item inside the category.

Layer	Responsibility	Examples
`models/staging/`	Typed interfaces to the synthetic upstream data	Stock, purchase lines, in-transit goods, demand families
`models/core/`	Shared dimensions used by the calculation	ISO calendar and item categories
`models/intermediate/scm/`	Reusable business transformations and recursive projections	Opening stock, ETA rules, ordered demand, item/category recursion

Layer	Responsibility	Examples
`models/marts/`	Final, documented KPI outputs	Item and category Weeks of Stock

The demand, receipt and projection models originally lived partly under `rep_`. They are intermediate data products here because they prepare and calculate reusable business logic. The two final KPI models remain consumption-facing. See the original-to-portfolio model map.

What makes the result trustworthy

- **Grain is explicit:** item or category × inventory channel × projection week. Composite uniqueness checks protect those grains.
- **The calendar is shared:** sequential week numbers measure distance across ISO year boundaries.
- **Receipts follow operational rules:** container ETA precedence, placeholder-date fallback, weekend adjustments and snapshot cutoff are retained.
- **Balances stay inspectable:** signed balances preserve the recursive calculation; zero-floored balances are separate presentation fields.
- **Tests check meaning:** hand-calculated examples cover the dynamic denominator, baseline, replenishment, category aggregation and known edge behavior, alongside structural dbt tests.
- **Definitions and consumers are visible:** YAML column documentation and a dbt exposure connect the final marts to this case study.

The original YAML provided a starting point for documentation and grain tests. This portfolio adds executable scenario checks and CI. Those additions are not a claim about the historical production CI setup.

Run the project

Python 3.12; no cloud credentials. Run from the repository root:

```
python -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
dbt deps --profiles-dir .
python scripts/build.py
```

The script seeds the synthetic source interfaces, builds and tests the models, generates dbt documentation, checks the documented SQL adaptations, verifies the example results, regenerates the figures and proves a deliberate dynamic-divisor error is caught. On Windows, activate with `.venv\Scripts\Activate.ps1`.

Explore lineage with `dbt docs serve --profiles-dir .`. For an immediate view without running anything, open the checked-in example results or read the stakeholder walkthrough.

Boundaries that matter

The implementation calls the **last positive-stock week** its depletion week. A later replenishment can therefore move it beyond an earlier shortage. It should not be labelled a first-stockout alert. The baseline has its own window convention, zero denominators use `DIV0`, and never-positive keys can disappear at the calendar join. These behaviors are preserved and tested rather than silently changed. Details and adoption considerations.

The example has 24 complete demand weeks, while KPI output is limited to positions 0–15. That gives every displayed six-week window enough input data. Stock still positive at the end of the supplied plan has a horizon bounded by that plan.

About this version

Original KPI modelling and SQL: **Filippo Carraro**. Portfolio work includes source reconstruction, synthetic scenarios, layer organisation, documented local execution adaptations, additional tests and explanatory figures. No historical business impact is invented for this demonstration.

The core calculations are source-derived rather than a replacement algorithm. Provenance, exact diffs and execution adaptations make that distinction reviewable. The local runner is configured for DuckDB; the original models used Snowflake syntax, and this repository does not claim a tested Snowflake deployment.

Publishing instructions · Validation details

Technical references: dbt data tests, dbt-utils, DuckDB recursive CTEs.